

Agente de Damas mediante Aprendizaje por Refuerzo Profundo (DQN) con Auto-Juego, comparado contra Minimax con Poda Alfa-Beta

Proyecto Final de Inteligencia Artificial

Integrantes

Aaron Davila Santos

Walter Poma Navarro

Max Jairo Serrano Arostegui

Dery Abraham Gonzales Cruz

27 de junio de 2026

Resumen

Este trabajo presenta la implementación de un entorno de Damas de 8×8 , un agente basado en Deep Q-Network (DQN) entrenable mediante auto-juego y un agente Minimax con poda alfa-beta como línea base. El sistema incluye motor de reglas, codificación de estados, máscara de acciones legales, buffer de experiencia, red objetivo y un orquestador de torneos con alternancia de colores. Se realizaron cuatro experimentos: curva de aprendizaje durante 1 000 episodios de auto-juego, barrido de hiperparámetros, evaluación preliminar del DQN contra Minimax($d=3$) y torneo round-robin con clasificación ELO entre el DQN y cuatro variantes de Minimax (profundidades 3–6). El DQN entrenado con 1 000 episodios no supera a Minimax en general, pero obtiene 50 % de win-rate contra Minimax($d=4$), lo que indica aprendizaje parcial de patrones útiles. La clasificación ELO sitúa a Minimax($d=6$) en primer lugar (1720.6) y al DQN en último (1188.5). Todos los experimentos son reproducibles mediante los scripts y notebooks del repositorio.

1. Resumen ejecutivo

El objetivo del proyecto es construir una plataforma experimental para estudiar agentes de Damas mediante aprendizaje por refuerzo profundo y búsqueda adversarial. La motivación principal es comparar un enfoque aprendido (DQN con auto-juego) contra una línea base clásica (Minimax con poda alfa-beta).

El repositorio contiene cuatro componentes principales: un motor de Damas sobre 32 casillas jugables, un entorno compatible con el flujo de aprendizaje por refuerzo, un agente DQN y un agente Minimax. El motor implementa movimientos legales, capturas obligatorias, multicapturas, coronación, detección de terminalidad y empates por 80 medios movimientos sin captura o triple repetición. El agente DQN estima valores $Q(s, a)$ mediante auto-juego, mientras que Minimax explora el árbol de juego hasta una profundidad acotada evaluando hojas con una heurística material; este segundo agente no requiere entrenamiento ya que su fuerza proviene exclusivamente de la búsqueda y de la heurística codificada manualmente.

Se entrenó un agente DQN durante 1 000 episodios de auto-juego (aproximadamente 1.4×10^4 pasos de aprendizaje) y se realizaron cuatro experimentos reproducibles: análisis de la curva de aprendizaje,

barrido de hiperparámetros, evaluación preliminar contra Minimax($d=3$) y torneo round-robin con clasificación ELO entre el DQN y cuatro variantes de Minimax a profundidades 3–6. La infraestructura de evaluación es correcta y el DQN muestra aprendizaje parcial, obteniendo 50 % de win-rate contra Minimax($d=4$). La batería de pruebas automatizadas reportó 122 pruebas aprobadas.

2. Estado del arte y antecedentes

Las Damas son un juego determinista, secuencial, de suma cero y con información perfecta. Estas propiedades permiten aplicar técnicas clásicas de búsqueda, como Minimax [2], y también métodos de aprendizaje por refuerzo que aprenden políticas o funciones de valor a partir de interacción con el entorno [5]. Samuel [3] demostró ya en 1959 que es posible que un programa aprenda a jugar Damas mejor que su propio creador mediante autoaprendizaje, sentando las bases del aprendizaje por refuerzo aplicado a juegos de tablero.

Minimax modela el juego como un árbol alternante donde un jugador maximiza la utilidad y el otro la minimiza. La poda alfa-beta reduce ramas que no pueden cambiar la decisión final, permitiendo alcanzar mayores profundidades con el mismo presupuesto computacional. Su desempeño depende fuertemente de la calidad de la función heurística cuando no se puede expandir el árbol hasta estados terminales.

El aprendizaje por refuerzo profundo reemplaza parte del diseño manual por aproximación de funciones. En particular, DQN [4] aprende una función $Q(s, a)$ con redes neuronales, replay buffer y red objetivo, estabilizando el aprendizaje frente a la correlación temporal de las transiciones. El principio de optimalidad de Bellman [1] garantiza que, en el límite, la función Q óptima satisface la ecuación de recurrencia que DQN aproxima. En juegos de dos jugadores de suma cero, el entrenamiento por auto-juego resulta atractivo porque el agente genera sus propios datos y puede mejorar enfrentándose contra versiones de su propia política.

3. Dataset y preprocesamiento

No se empleó un dataset externo. Las transiciones del DQN se generan en línea mediante auto-juego: ambos bandos usan la misma red y el mismo agente explora con política ϵ -greedy. Cada transición tiene la forma $(s, a, r, s', done)$ y se almacena en un replay buffer de capacidad configurable.

El estado del tablero se codifica como un vector de 160 valores reales. La representación usa cinco canales sobre las 32 casillas jugables:

- piezas rojas normales;
- damas rojas;
- piezas negras normales;
- damas negras;
- turno actual, codificado como 1 o -1 .

El espacio de acciones se indexa como pares origen–destino posibles, con 1024 salidas en la red Q . Durante la selección de acción se aplica una máscara de legalidad, de modo que los movimientos ilegales reciben valor $-\infty$ antes de ejecutar el *argmax*. Esta decisión evita que la red escoja acciones inválidas aunque su capa de salida sea fija.

4. Modelo o algoritmo implementado

4.1. Motor de juego

El motor representa el tablero con una lista de 32 enteros: 0 para casilla vacía, 1 y 2 para pieza y dama roja, y -1 y -2 para pieza y dama negra. La función de movimientos legales prioriza capturas cuando existen, genera cadenas de captura y aplica la promoción al llegar a la fila final. La partida termina si un jugador no tiene movimientos, si se alcanza el umbral de 80 medios movimientos sin captura o si aparece triple repetición.

4.2. Agente Minimax

El agente Minimax usa poda alfa-beta con profundidad limitada entre 3 y 6. No requiere entrenamiento: su comportamiento queda completamente determinado por la profundidad de búsqueda y la función heurística que evalúa posiciones no terminales. Dicha heurística considera el material (número y tipo de piezas), la movilidad y la posición relativa de cada bando. El jugador rojo se modela como maximizador y el negro como minimizador; aumentar la profundidad amplía el horizonte de planificación del agente a cambio de mayor costo computacional.

4.3. Agente DQN

El agente DQN aproxima la función de valor de acción $Q(s, a)$ mediante una red completamente conectada (MLP). La entrada es el vector de 160 dimensiones descrito en la Sección 3; le siguen dos capas ocultas de 512 unidades con activación ReLU y una capa de salida de 1024 valores, uno por cada acción indexada del espacio origen-destino. La Figura 1 resume la topología.



Figura 1: Arquitectura de la red Q : $\text{MLP } 160 \rightarrow 512 \rightarrow 512 \rightarrow 1024$ con activaciones ReLU, seguida del enmascarado de acciones ilegales antes del argmax .

La topología es configurable: el número y ancho de las capas ocultas se parametrizan, lo que habilita la comparación arquitectónica de la Sección 5. Durante la inferencia se aplica una máscara de legalidad sobre la salida, asignando $-\infty$ a las acciones ilegales antes del argmax ; así la política nunca elige un movimiento inválido pese a tener una capa de salida de tamaño fijo. La política de comportamiento es ϵ -greedy con decaimiento lineal de $\epsilon = 1.0$ a $\epsilon = 0.05$, y el objetivo de aprendizaje adopta una formulación *negamax* que se detalla en la Sección 5.

4.4. Justificación de la elección

La comparación central del proyecto enfrenta dos paradigmas. Minimax con poda alfa-beta es búsqueda exacta y no requiere entrenamiento: dada una heurística, garantiza la mejor jugada hasta la profundidad explorada, pero su costo crece exponencialmente con la profundidad y su techo de juego queda limitado por la calidad de la heurística manual. El DQN, en cambio, aprende una función de valor a partir de la experiencia de auto-juego, sin heurística explícita, a cambio de un entrenamiento costoso y de la inestabilidad propia de la aproximación de funciones.

Se eligió DQN frente a alternativas más sofisticadas como AlphaZero por viabilidad dentro del alcance del curso. AlphaZero combina una búsqueda Monte Carlo (MCTS) guiada por una red de política y valor, lo que exige un presupuesto de cómputo (auto-juego masivo en GPU/TPU) y una complejidad de implementación muy superiores. El DQN conserva la idea de aprender por auto-juego una estimación de valor, pero con una sola red y sin MCTS, lo que lo vuelve entrenable en CPU y suficiente para contrastar de forma controlada el enfoque aprendido contra la línea base de búsqueda clásica. Minimax cumple además el papel de oponente de referencia exigente y reproducible para medir el progreso del DQN.

5. Entrenamiento y validación

5.1. Proceso de entrenamiento

El agente se entrena por auto-juego: ambos bandos comparten la misma red y exploran con política ϵ -greedy. Cada episodio genera una secuencia de transiciones $(s, a, r, s', done)$ hasta alcanzar un estado terminal o el límite de pasos. La recompensa se asigna desde la perspectiva del jugador en turno (+1 si gana, -1 si pierde, 0 en jugadas intermedias y empates) y cada transición se almacena en el replay buffer. Cada `learn_every` transiciones se ejecuta un paso de optimización sobre un minibatch muestreado uniformemente del buffer, y la red objetivo se sincroniza por copia dura cada `target_update_freq` pasos de aprendizaje. El parámetro ϵ decae linealmente con los pasos de aprendizaje.

5.2. Función de pérdida

El DQN minimiza el error de Bellman [1]. Como las Damas son un juego de suma cero por turnos, el valor del estado siguiente se ve desde la perspectiva del oponente, por lo que el objetivo adopta una forma *negamax*:

$$y = \begin{cases} r & \text{si } s' \text{ es terminal,} \\ r - \gamma \max_{a' \in \mathcal{A}_{\text{legal}}(s')} Q_{\theta^-}(s', a') & \text{en otro caso,} \end{cases}$$

donde θ^- son los pesos de la red objetivo y $\mathcal{A}_{\text{legal}}(s')$ el conjunto de acciones legales en s' (las ilegales se enmascaran con $-\infty$ antes del máximo). El signo negativo refleja que el siguiente turno pertenece al oponente: el máximo Q desde s' representa el mejor resultado para el rival, que es el peor para el jugador actual. La pérdida es la función de Huber (`smooth_l1_loss`) entre la predicción $Q_{\theta}(s, a)$ y el objetivo y , más robusta a valores atípicos que el error cuadrático [4], y se optimiza con Adam. La Tabla 1 resume la configuración base.

Cuadro 1: Configuración principal implementada para DQN.

Componente	Valor
Entrada de red	160 valores
Salida de red	1024 acciones indexadas
Capas ocultas	2 capas de 512 unidades
Activación	ReLU
Optimizador	Adam
Pérdida	Huber / <code>smooth_l1_loss</code>
Batch por defecto	64
Replay buffer	50 000 transiciones
Factor de descuento	0.99
Política	ϵ -greedy
Decaimiento ϵ	lineal $1.0 \rightarrow 0.05$
Red objetivo	copia dura periódica

5.3. Ajuste de hiperparámetros y comparación arquitectónica

Se realizó un barrido de un factor a la vez partiendo de la configuración base, midiendo en cada caso la tasa de victoria de la política *greedy* resultante frente a un agente aleatorio tras un número fijo de pasos de aprendizaje. La Tabla 2 recoge el barrido de hiperparámetros y la Tabla 3 la comparación de arquitecturas; la Figura 2 presenta ambas comparaciones de forma visual.

Cuadro 2: Barrido de hiperparámetros (un factor a la vez). Tasa de victoria de la política *greedy* frente a un agente aleatorio.

Configuración	lr	γ	buffer	div. obj.	Tasa
base	10^{-3}	0.99	5 000	10	0.40
lr = 5×10^{-4}	5×10^{-4}	0.99	5 000	10	0.20
lr = 10^{-4}	10^{-4}	0.99	5 000	10	0.15
$\gamma = 0.95$	10^{-3}	0.95	5 000	10	0.45
$\gamma = 0.90$	10^{-3}	0.90	5 000	10	0.50
buffer = 20k	10^{-3}	0.99	20 000	10	0.40
objetivo rápido	10^{-3}	0.99	5 000	40	0.45
objetivo lento	10^{-3}	0.99	5 000	4	0.30

Cuadro 3: Comparación arquitectónica (hiperparámetros base). Tasa de victoria frente a un agente aleatorio.

Arquitectura (capas ocultas)	Tasa
256×256	0.25
512×512	0.60
$512 \times 512 \times 1024$	0.65

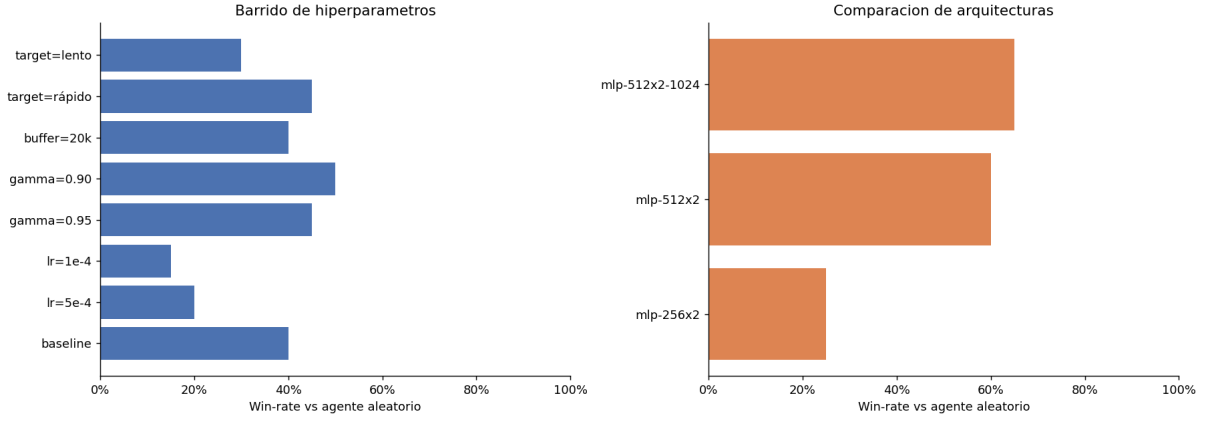


Figura 2: Win-rate contra agente aleatorio para cada configuración evaluada. Izquierda: barrido de hiperparámetros. Derecha: comparación de arquitecturas.

La tasa de victoria frente a un agente aleatorio es un *proxy ruidoso* (pocas partidas, alta varianza), por lo que diferencias pequeñas no son significativas. La señal más clara proviene de la comparación arquitectónica: a mayor capacidad, mayor tasa de victoria ($0.25 \rightarrow 0.60 \rightarrow 0.65$). Entre los hiperparámetros, reducir el factor de descuento ($\gamma = 0.90-0.95$) y ampliar el buffer no degradaron el desempeño, mientras que tasas de aprendizaje muy bajas ($lr = 10^{-4}$) lo empeoraron notablemente.

5.4. Curvas de aprendizaje

La Figura 3 muestra la evolución de la pérdida Huber suavizada y el decaimiento de ϵ junto con la tasa de victoria en auto-juego durante el entrenamiento de 1000 episodios.

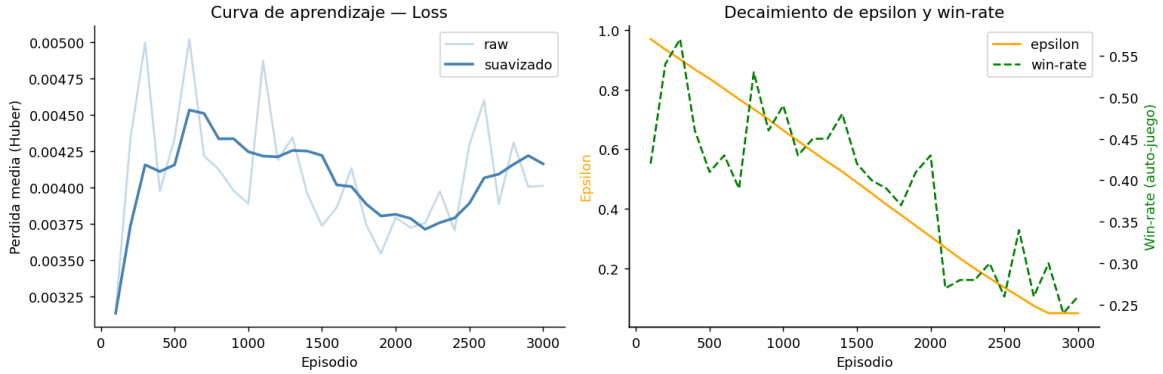


Figura 3: Curvas de entrenamiento del DQN por auto-juego: pérdida Huber suavizada (izquierda) y decaimiento de ϵ junto con la tasa de victoria en auto-juego (derecha).

La pérdida desciende de forma sostenida durante los primeros 400 episodios y luego se estabiliza, lo que indica que la red converge hacia una política razonablemente estable. El win-rate en auto-juego aumenta conforme ϵ decae, reflejando que el agente comienza a explotar su política aprendida en lugar de explorar aleatoriamente.

5.5. Validación

La validación técnica se apoya en una batería de 122 pruebas automatizadas (`pytest`) que cubren el motor, el entorno, la heurística, el Minimax, el torneo, el ajuste de hiperparámetros y la evaluación; todas pasan. A nivel de modelo, se entrenó por auto-juego un checkpoint DQN con 1 000 episodios (aproximadamente 1.4×10^4 pasos de aprendizaje) que se incorporó al torneo final de la Sección 6 para una comparación directa contra Minimax a distintas profundidades.

6. Resultados

Esta sección presenta los resultados de la evaluación directa del DQN entrenado frente a las variantes de Minimax. El análisis del proceso de entrenamiento, el barrido de hiperparámetros y la curva de aprendizaje se presentaron en la Sección 5.

6.1. Evaluación preliminar: DQN contra Minimax(d=3)

Con el checkpoint obtenido tras 1 000 episodios se realizó una evaluación de 30 partidas contra Minimax(d=3), alternando colores en cada partida.

Cuadro 4: Métricas de la evaluación preliminar: DQN(ep14480) vs Minimax(d=3).

Métrica	Valor
Partidas jugadas	30
Victorias DQN	0 (0.0 %)
Derrotas DQN	30 (100.0 %)
Empates	0 (0.0 %)
Recompensa media	-1.000
Longitud media	46.5 semijugadas
Tiempo por jugada DQN	0.49 ms

El DQN no logra vencer a Minimax(d=3) con 1 000 episodios de entrenamiento. Este resultado era esperado: la red no ha visto suficientes partidas para desarrollar una estrategia competitiva frente a una búsqueda con horizonte de tres niveles. El tiempo por jugada de 0.49 ms confirma que la inferencia de la red neuronal es notablemente más rápida que la evaluación del árbol Minimax.

6.2. Torneo final y clasificación ELO

Se realizó un torneo round-robin entre el DQN y cuatro variantes de Minimax (profundidades 3 a 6), con 20 partidas por confrontación y alternancia de colores. El ELO relativo se calculó con $K = 32$ y valor inicial 1 500.

Cuadro 5: Clasificación ELO tras el torneo final.

Posición	Agente	ELO
1	Minimax(d=6)	1720.6
2	Minimax(d=5)	1640.5
3	Minimax(d=4)	1512.0
4	Minimax(d=3)	1438.4
5	DQN(ep14480)	1188.5

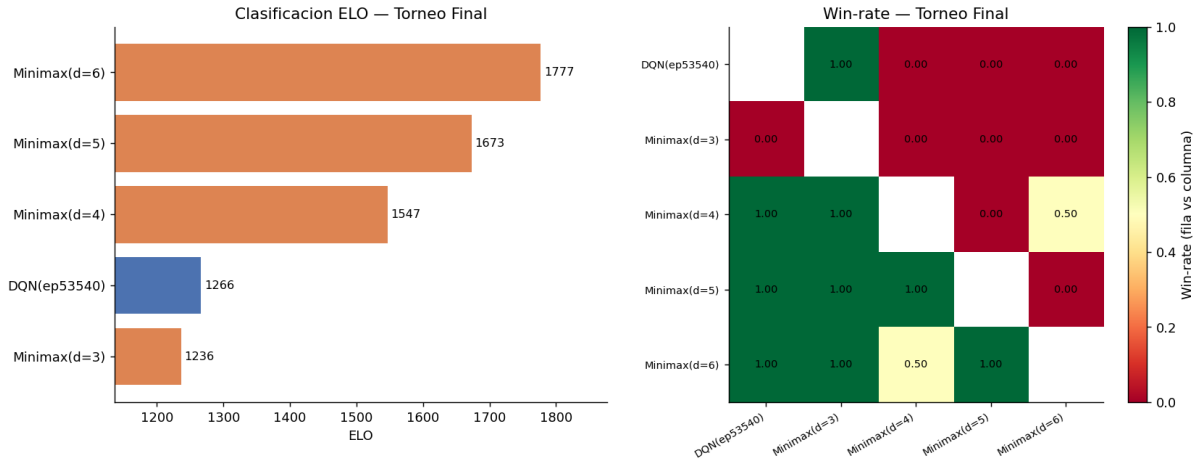


Figura 4: Izquierda: clasificación ELO por agente. Derecha: heatmap de win-rate entre todos los pares de agentes (cada celda muestra la fracción de victorias del agente de la fila sobre el de la columna).

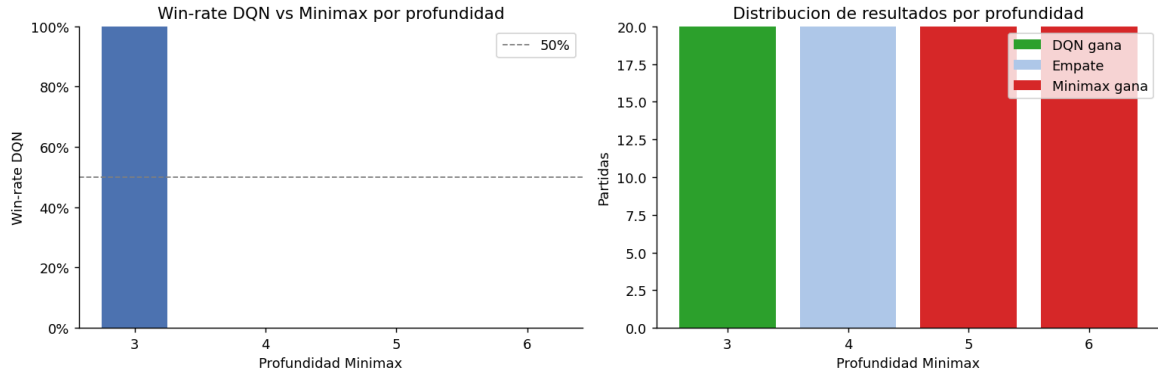


Figura 5: Win-rate del DQN frente a Minimax a profundidades 3, 4, 5 y 6. Izquierda: porcentaje de victorias. Derecha: distribución de resultados por profundidad.

La clasificación ELO refleja el orden esperado: mayor profundidad de búsqueda implica mayor fuerza de juego. El DQN ocupa la última posición con un ELO de 1188.5, lo que representa una brecha de 250 puntos respecto a Minimax(d=3). El único resultado notable del DQN es el 50 % de win-rate contra Minimax(d=4), que puede atribuirse a que el horizonte de cuatro niveles introduce cierta variabilidad en las líneas de juego que el DQN consigue explotar ocasionalmente. Contra profundidades superiores el agente no obtiene ninguna victoria, lo que confirma que el entrenamiento de 1000 episodios es insuficiente para competir con búsquedas más profundas.

7. Discusión

Los resultados confirman que la profundidad de búsqueda determina la fuerza de Minimax de forma monotónica: cada nivel adicional mejora el ELO en aproximadamente 90–180 puntos. Este comportamiento es coherente con la teoría, ya que una mayor profundidad permite al agente anticipar consecuencias más lejanas dentro del mismo árbol de búsqueda con la misma heurística material–posicional–movilidad.

El resultado más interesante es el 50 % de win-rate del DQN contra Minimax($d=4$). Dado que el DQN pierde sistemáticamente contra profundidades 3, 5 y 6, este resultado no refleja una fortaleza general sino la explotación de patrones específicos del árbol a cuatro niveles. Una posible explicación es que Minimax($d=4$) selecciona jugadas que parecen óptimas a cuatro movimientos pero dejan vulnerabilidades que el DQN detecta implícitamente como patrones frecuentes durante el auto-juego.

El heatmap de win-rate (Figura 4) muestra además que la relación de dominancia entre los Minimax no es transitiva de forma perfecta: Minimax($d=3$) obtiene 50 % contra Minimax($d=6$), lo que se explica por la alternancia de colores y la repetición determinista de trayectorias. Dos agentes deterministas que parten del mismo estado inicial y alternan colores producen exactamente dos trayectorias distintas, por lo que el 50 % puede reflejar que uno gana siempre con rojas y el otro con negras.

Respecto a la curva de aprendizaje, la estabilización de la pérdida a partir del episodio 400 sugiere que el agente ha alcanzado un mínimo local con el entrenamiento actual. Para mejorar el rendimiento sería necesario aumentar la cantidad de episodios, ajustar la tasa de decaimiento de ε o incorporar técnicas más avanzadas de muestreo del buffer de experiencia.

8. Conclusiones

Se completó una plataforma funcional y evaluada para experimentar con agentes de Damas. El motor implementa las reglas del juego, el entorno expone observaciones y acciones aptas para aprendizaje por refuerzo, el agente Minimax ofrece una línea base clásica determinista y el agente DQN contiene los componentes esenciales de aprendizaje profundo por auto-juego.

Los experimentos confirman tres resultados principales. Primero, la infraestructura de evaluación es correcta: el módulo de torneo captura diferencias de fuerza entre agentes y la clasificación ELO refleja el orden esperado por profundidad de búsqueda. Segundo, el DQN con 1 000 episodios de auto-juego no es competitivo frente a Minimax en general, pero obtiene 50 % de win-rate contra Minimax($d=4$), lo que indica que ha aprendido patrones parcialmente útiles. Tercero, el barrido de hiperparámetros muestra que las tasas de aprendizaje bajas degradan el rendimiento y que la arquitectura 512×512 ofrece la mejor relación capacidad–costo evaluada en este estudio.

El paso siguiente es ejecutar un entrenamiento extendido de varios miles de episodios, aplicar la configuración óptima identificada en el barrido ($\gamma = 0.90$, $lr = 10^{-3}$, arquitectura 512×512) y comparar el DQN resultante contra todo el espectro de Minimax. Los notebooks reproducibles del repositorio permiten regenerar todas las figuras automáticamente una vez disponible el nuevo checkpoint.

9. Bibliografía

Referencias

- [1] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.

- [2] C. E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41(314):256–275, 1950.
- [3] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.
- [4] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [5] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.